



Centro Regional Universitario Córdoba IUA
Ing. en Informática

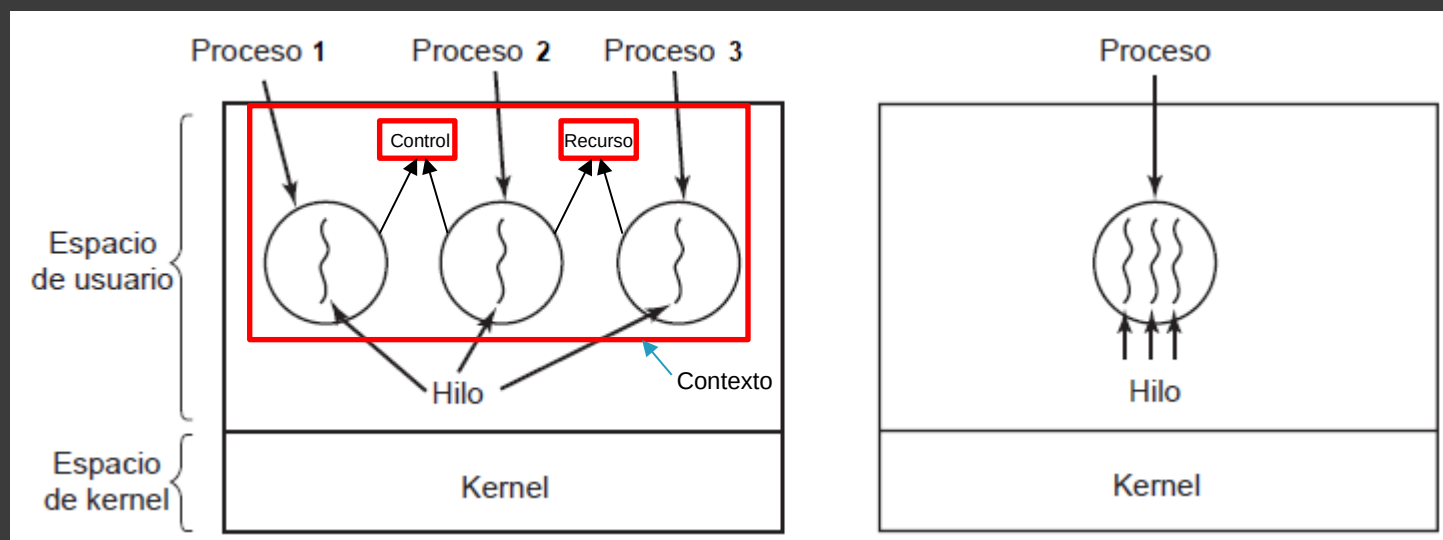
SISTEMAS EN TIEMPO REAL

Mag. Ing. José Ducloux
jducloux@iua.edu.ar

2026

POSIX en C

- Hilos
- Procesos
- Sincronización de hilos
- **Sincronización de procesos**



Sincronización de procesos

- Más complicada que los hilos
 - Los procesos pueden “vivir” libremente en cualquier máquina, con cualquier sistema operativo, ubicado en cualquier lugar dentro de una red tan grande como Internet.
- Sincronización dentro de un solo host con POSIX
 - Mecanismos de control dentro de una sola máquina.
 - Varios procesos ejecutándose concurrentemente con acceso simultáneo a recursos compartidos.
 - Los procesos se ejecutan en el contexto de un mismo sistema operativo.

Sincronización de procesos en un solo host

- Mecanismos de control proporcionados por POSIX
 - **Semáforos con nombre:**
 - se pueden usar globalmente en todo el sistema.
 - **Mutex con nombre:**
 - Se pueden usar globalmente en todo el sistema.
 - Estos mutex deben colocarse dentro de una memoria compartida para que estén disponibles para múltiples procesos.
 - **Variables de condición con nombre:**
 - Se pueden usar globalmente en todo el sistema.
 - Éstas deben colocarse dentro de una memoria compartida para que estén disponibles para múltiples procesos.
- El funcionamiento de cada mecanismo es similar al de hilos, sólo que son visibles globalmente en todo el sistema.

Semáforos con nombre

- Funciones
 - `sem_open()`: abrir e inicializar el semáforo.
 - `sem_close()`: cerrar el semáforo.
 - `sem_unlink()`: desvincular al semáforo.
 - `sem_wait()` y `sem_post()`: entrar y salir de la sección crítica.

Semáforos con nombre

- Ejemplo 1
 - Resolver la carrera de datos del ejemplo 6 de la presentación de procesos.
 - Uso de la API de bifurcación y memoria compartida.
 - La variable *counter* se pone en la región de memoria compartida.
 - El proceso padre crea un hijo y ambos intentan incrementar la variable compartida *counter*.
 - Hay momentos en donde el proceso padre y el hijo actualizan la variable sin obtener el último valor modificado.
 - Se imprime 1 en ambos procesos demostrando la carrera de datos.
 - El uso de un semáforo permite el acceso a la memoria compartida sólo a uno de los dos procesos, eliminando la carrera de datos.

Semáforos con nombre

- Ejemplo 1
 - A veces, el proceso hijo obtiene primero la CPU e incrementa el contador, y a veces el proceso padre lo hace. Aún existe una condición de carrera.
 - Ambos ya no ingresan a la sección crítica al mismo tiempo y, por lo tanto, satisfacen la integridad de los datos compartidos.

Mutex con nombre

- El mutex debe definirse dentro de un lugar que sea accesible para todos los procesos.
- La mejor opción es una región de memoria compartida.
- Funciones
 - `pthread_mutex_init()`
 - `pthread_mutexattr_setpshared()`
 - `pthread_mutex_destroy()`
 - `pthread_mutex_lock()`
 - `pthread_mutex_unlock()`

Mutex con nombre

- Ejemplo 2
 - Es similar al ejemplo 1 resolviendo la carrera de datos utilizando mutex con nombre.
 - Se crea una región de memoria compartida para almacenar el mutex.
 - Dado que cada objeto de memoria compartida es global, el mutex almacenado allí podrá ser accedido por otros procesos en todo el sistema.

Mutex con nombre

- Ejemplo 2
 - Se declaran:
 - Un descriptor de archivo global para apuntar a una región de memoria compartida destinada a almacenar la variable compartida (counter).
 - Un descriptor de archivo global para la región de memoria compartida que almacena el mutex compartido.
 - Un puntero a la variable compartida.
 - Un puntero al mutex compartido.

Mutex con nombre

- Ejemplo 2
 - Se crea un objeto de memoria /mutex0 para compartir el mutex.
 - Luego se inicializa el mutex con atributos que indiquen que debe ser compartido.
 - La sección crítica está en la función `inc_counter()`, en la cual se utiliza el mutex compartido.

Mutex con nombre

- Ejemplo 3
 - Las memorias compartidas y los mutex con nombre se pueden utilizar en todo el sistema mediante cualquier proceso. No es obligatorio tener un proceso bifurcado para poder utilizar estos objetos.
 - Este ejemplo muestra cómo se puede usar un mutex compartido y una memoria compartida (bandera) para finalizar simultáneamente varios procesos que se están ejecutando al mismo tiempo. Todos los procesos terminan después de presionar ctrl+c en uno de ellos.
 - El primer proceso crea el objeto de memoria compartida para la bandera compartida, y el objeto de memoria del mutex con nombre.
 - Se utilizan señales (interrupciones por software).

Mutex con nombre

- Ejemplo 3
 - En la función `main()` se inicializa la bandera compartida, y el mutex compartido. Luego el proceso entra en una espera activa (`while`) hasta que la bandera de cancelación sea verdadera.
 - Al presionar `ctrl+c` se envía la señal de interrupción `SIGINT` que recibe el proceso en primer plano que se ejecuta en una terminal. La función `sigint_handler()` se llama cada vez que se recibe la señal `SIGINT`.

Variables de condición con nombre

- De manera similar a los mutex con nombre, se necesita asignar una variable de condición con nombre en una región de memoria compartida para usarla en un sistema con varios procesos.
- Cada variable de condición debe usarse junto con un mutex complementario que la proteja.

Variables de condición con nombre

- Ejemplo 4
 - Al ejecutar el programa, se le debe pasar como parámetro un número entero mayor o igual a 1, el cual usará el proceso como límite superior para comenzar a contar de manera ascendente. Un proceso sólo podrá imprimir su valor final de conteo una vez que el proceso antecesor, con un número más pequeño, haya contado. El proceso con el número 1 será el primero en contar, incluso si se ejecuta al último.
 - El primer proceso en ejecutarse crea las regiones de memoria compartida; el resto las abren.
 - Se tienen 3 regiones de memoria compartida
 - Una para el contador compartido.
 - Una para la variable de condición con nombre.
 - Una para el mutex con nombre.



Centro Regional Universitario Córdoba IUA
Ing. en Informática

¿Preguntas?